

SIMATS ENGINEERING

CO4 ASSESSMENT TOOL 1

NAME : JAIVANT.S

REG NO : 192311326

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE : CSA0613

Title:

Coin Change Problem Using Dynamic Programming

Objective

To determine the minimum number of coins required to make a given target amount using the available coin denominations through Dynamic Programming.

Problem Statement

A cashier must provide change for a given amount using a set of available coin denominations. The objective is to determine the minimum number of coins required to make the exact amount. Each denomination can be used multiple times.

Algorithm

1. Read the number of coin types.
2. Read the coin denominations.
3. Read the target amount.
4. Create a DP array of size amount + 1.
5. Initialize all values with infinity.
6. Set $dp[0] = 0$.
7. For every amount, check all available coins.
8. Update the minimum number of coins.
9. Display the result.

Source Code:

```
main.py Visualize Run
1 n = int(input("Enter number of coin types: "))
2
3 coins = list(map(int, input("Enter coin denominations: ").split()))
4
5 amount = int(input("Enter target amount: "))
6
7 dp = [float('inf')] * (amount + 1)
8 dp[0] = 0
9
10 for i in range(1, amount + 1):
11     for coin in coins:
12         if coin <= i:
13             dp[i] = min(dp[i], dp[i - coin] + 1)
14
15 if dp[amount] == float('inf'):
16     print("Amount cannot be formed")
17 else:
18     print("Minimum Coins =", dp[amount])
```

Output:

```
Output
Enter number of coin types: 3
Enter coin denominations: 1 2 5
Enter target amount: 11
Minimum Coins = 3

=== Code Execution Successful ===
```

Analysis

For the target amount 11, the possible optimal combination is:

$$5 + 5 + 1 = 11$$

Therefore, the minimum number of coins required is:

3

The Dynamic Programming approach stores the minimum number of coins required for every smaller amount and uses those results to calculate the solution efficiently.

Time Complexity

$O(n \times \text{amount})$

Space Complexity

$O(\text{amount})$

Conclusion

The Coin Change Problem was successfully implemented using Dynamic Programming. The program correctly determines the minimum number of coins required to form the given target amount.
